CATEGORY C

Part A — Coding Problems

All the Part ‘A’ Coding Problems has been solved and uploaded in GitHub.
Part ‘B’ and ‘C’ have been solved here in this document

Problem 1: Vowel and Consonant Counter

TASK

Write a program that reads a single word and then counts and prints the total number of vowels and consonants in that word. Assume the input word contains only alphabetic characters (case-insensitive).

EXAMPLE

```
Sample Input: Programming

Expected Output: Vowels: 3
                 Consonants: 8
```

HINT

Iterate through each character of the word. Use conditional statements to check if a character is a vowel. Remember that vowels are 'a', 'e', 'i', 'o', 'u' (case-insensitive). All other alphabetic characters are consonants.

Problem 2: Word Reverser and Palindrome Checker

TASK

Create a program that reads a word, reverses it using a mutable text builder (like StringBuilder in Java), and then prints the reversed word. Additionally, determine and print whether the original word reads the same forwards and backwards (is a palindrome).

EXAMPLE

```
Sample Input: level
              java

Expected Output: level - palindrome
                 avaj - not a palindrome
```

HINT

Use a mutable string class to build the reversed word. Compare the original string with the reversed string to check for palindrome property. Remember to consider case sensitivity if not specified otherwise.

Problem 3: Even and Odd Numbers in a List

TASK

Given a list of integers, write a program to count how many numbers in the list are even and how many are odd. Print the counts for both categories.

EXAMPLE

```
Sample Input: [3, 8, 12, 5, 7, 10]

Expected Output: Even: 3
                 Odd: 3
```

HINT

Iterate through each number in the list. Use the modulo operator (%) to determine if a number is even or odd.

Problem 4: In-Place Array Reversal

TASK

Write a program that reverses the order of token numbers stored in a list. The reversal must be done 'in-place', meaning you should modify the original list directly without creating a second list.

EXAMPLE

```
Sample Input: [11, 22, 33, 44]

Expected Output: [44, 33, 22, 11]
```

HINT

Use two pointers, one starting from the beginning and one from the end of the array. Swap the elements at these pointers and move them towards the center until they meet or cross.

Problem 5: Digit Sum and Number Reversal

TASK

Write a program that reads a positive integer and performs two operations: it calculates and prints the sum of its digits, and it prints the number with its digits reversed.

EXAMPLE

```
Sample Input: 4721

Expected Output: Sum of digits: 14
                 Reverse: 1274
```

HINT

To extract digits, repeatedly use the modulo operator (%) by 10 to get the last digit and integer division (/) by 10 to remove the last digit. Build the reversed number by multiplying the current reversed number by 10 and adding the extracted digit.

Problem 6: Prime Number Checker

TASK

Write a program that reads a number greater than 1 and determines whether it is a prime number. A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. Print whether the number is prime or not.

EXAMPLE

```
Sample Input: 29
              21

Expected Output: 29 is prime
                 21 is not prime
```

HINT

To check for primality, iterate from 2 up to the square root of the number. If any number in this range divides the input number evenly, it's not prime.

Problem 7: Factorial Calculation

TASK

Write a program that reads a non-negative integer and calculates its factorial. The factorial of a non-negative integer 'n', denoted by 'n!', is the product of all positive integers less than or equal to 'n'. The factorial of 0 is defined as 1.

EXAMPLE

```
Sample Input: 5
              0

Expected Output: 120
                 1
```

HINT

Use a loop to multiply numbers from 1 up to the input number. Handle the special case for 0 factorial.

Problem 8: Student Grade Assignment

TASK

Write a program that reads a student's marks (out of 100) and prints their corresponding grade based on the following criteria:

- A: 90 or above
- B: 75 to 89
- C: 60 to 74
- D: 40 to 59
- F: Below 40

EXAMPLE

```
Sample Input: 72
              95
              35

Expected Output: Grade C
                 Grade A
                 Grade F
```


HINT

Use an if-else if-else ladder to check the conditions in the correct order, starting from the highest grade boundary.

Problem 9: Leap Year Detector

TASK

Write a program that reads a year and determines if it is a leap year. A year is a leap year if it is divisible by 4, but not by 100, unless it is also divisible by 400.

EXAMPLE

```
Sample Input: 2024
              1900
              2000

Expected Output: Leap year
                 Not a leap year
                 Leap year
```

HINT

Formulate the conditions using logical operators (AND, OR) to implement the leap year rules. Check for divisibility by 4, 100, and 400 in the correct logical sequence.

Problem 10: Student Result Card Generator

TASK

Design a system for student result cards. Each student has a name and an array of three marks. Implement a method to calculate the student's average mark. Based on the average, assign a grade using the following conditions: 'B' for an average of 75 or more, 'D' for an average between 40 and 59 (inclusive), and other grades as per your design. Finally, print the student's name in capital letters, their average mark (one decimal place), and their assigned grade.

EXAMPLE

```
Sample Input: Asha [80, 90, 70]
              Ravi [60, 55, 50]

Expected Output: ASHA: Average 80.0, Grade B
                 RAVI: Average 55.0, Grade D
```

HINT

Create a Student class with fields for name and marks array. Implement a method within the Student class to calculate the average. Use conditional logic (if-else if-else) to determine the grade based on the average. Convert the student's name to uppercase for printing.

Part B — Quiz

Choose the best answer for each question.

Q1. Consider two String objects, s1 and s2, created as follows:

```
String s1 = new String("hello");
String s2 = new String("hello");
```

Which of the following statements is true regarding the comparison of s1 and s2 in Java? A.`s1 == s2` is true, and `s1.equals(s2)` is true.

B.`s1 == s2` is false, and `s1.equals(s2)` is false.

C.`s1 == s2` is true, and `s1.equals(s2)` is false.

D.`s1 == s2` is false, and `s1.equals(s2)` is true.

Answer -> D

Q2. Given the following sequence of operations on a StringBuilder object: `StringBuilder sb = new StringBuilder();`

```
sb.append("Java");
sb.append(" Lab");
sb.reverse();
```

Answer -> B

What will be the content of the StringBuilder object sb after these operations? A."Java Lab"

B."baL avaJ"

C."Lab Java"

D."avaL baJ"

Answer -> B

Q3. Consider a Student class with the following constructors:

```
class Student {
    String name;
    String department;

    public Student() {
        this("Unknown", "Not assigned");
    }

    public Student(String name) {
        this(name, "Not assigned");
    }

    public Student(String name, String department) {
        this.name = name;
        this.department = department;
    }
}
```

If the following code is executed:

```
Student s = new Student("Ravi");
```


Which constructor will be directly called by new Student("Ravi")?

A.`public Student()`

B.`public Student(String name)`

C.`public Student(String name, String department)`

D.The code will result in a compile-time error due to ambiguity.

Answer -> D

Q4. Consider a Wallet class with a balance field and an add(int amount) method. If w1 and w2 are Wallet reference variables, and the following operations occur:

```
Wallet w1 = new Wallet(500);
```

```
Wallet w2 = w1;
w2.add(200);
```

What will be the balance seen through w1 and w2 after w2.add(200) is executed? A.`w1` balance: 500, `w2` balance: 700

B.`w1` balance: 700, `w2` balance: 500

C.`w1` balance: 500, `w2` balance: 500

D.`w1` balance: 700, `w2` balance: 700

Answer -> C

Q5. Consider a Visitor class that tracks the total number of visitors using a static variable count.

```
class Visitor {
    private static int count = 0;
    private String name;

    public Visitor(String name) {
        this.name = name;
        count++; // Increment the shared counter
    }

    public static int getTotalVisitors() {
        return count;
    }
}
```

If three Visitor objects are created:

```
new Visitor("Asha");
new Visitor("Ravi");
new Visitor("Neha");
```

What will be the value returned by Visitor.getTotalVisitors() after these objects are created? A.0

B.1

C.3

D.Undefined, as static variables require an object instance.

Answer -> A

Q6. Consider an integer array named scores with elements [45, 78, 92, 60, 85]. If you iterate through this array using a standard for loop from index 0 to scores.length - 1 and print each element:

```
int[] scores = {45, 78, 92, 60, 85};
for (int i = 0; i < scores.length; i++) {
    System.out.print(scores[i] + " ");
}
```

What will be the output?

A.45 78 92 60 85

B.85 60 92 78 45

C.45 78 92 60

D.Error: Index out of bounds.

Answer -> D

Q7. Analyze the following for loop in Java:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.print(i + " ");
}
```

What will be the output when this code segment is executed? A.1 2 3 4 5

B.1 2 4 5

C.3

D.1 2

Answer -> B

Q8. Consider the following Java code snippet:

```
class MyClass {
    public static void change(int n) {
        n = 10;
    }

    public static void main(String[] args) {
        int x = 5;
        change(x);
        System.out.println(x);
    }
}
```

What will be printed to the console when the main method is executed? A.5

B.10

C.0

D.An error will occur because `n` is not initialized.

Answer -> A

Q9. Consider the following Java code:

```
int total = 250;
double count = 3;
```

```
double avg = total / count;
System.out.println(avg);
```

What will be the output when this code is executed?

A.83

B.83.0

C.83.33333333333333

D.83.33

Answer -> C

Q10. Consider a Payment abstract class with an abstract method pay() and a Card subclass that extends Payment and overrides the pay() method to include a 2% fee.

```
abstract class Payment {
    abstract void pay(double amount);
}

class Card extends Payment {
    @Override
    void pay(double amount) {
        double fee = amount * 0.02;
        System.out.println("Paid " + (amount + fee) + " by card");
    }
}
```

If the following code is executed:

```
Payment p = new Card();
p.pay(500);
```

What will be the output?

A.Paid 500.0 by card

B.Paid 510.0 by card

C.Error: Cannot instantiate abstract class Payment.

D.Error: Payment class does not have a concrete pay method.

Answer -> B

Q11. Consider the following Java code snippet and the conceptual object diagram: `String name = "java";`

```
String upperName = name.toUpperCase();
```

Which of the following statements accurately describes the state of the name object after the toUpperCase() method call, based on Java's String immutability?

A.The `name` object's value changes to "JAVA".

B.The `name` object's value remains "java", and a new `String` object "JAVA" is created for `upperName`. C.Both `name` and `upperName` refer to the same new `String` object "JAVA". D.The original `name` object is destroyed, and a new one is created.

Answer -> B

Q12. Given the Java code snippet and the conceptual object diagram, how many String objects are created in memory, considering the String pool?

```
String s1 = "hello";
String s2 = "hello";
String s3 = new String("hello");
```

A.1

B.2

C.3

D.0

Answer -> B

Q13. Consider the Word class and the code snippet. What will be the output of the following Java code?

```
class Word {
    private String text = "Campus";
    public char letterAt(int i) {
        return text.charAt(i);
    }
    public int getTextLength() {
        return text.length();
    }
}

Word myWord = new Word();
System.out.println(myWord.getTextLength());
System.out.println(myWord.letterAt(2));
```

A.6 / m

B.6 / p

C.5 / m

D.7 / p

Answer -> A

Q14. Given the Java code snippet, what will be the output?

```
String email = "user@domain.com";
int index = email.indexOf("@");
System.out.println(index);
```

A.4

B.0

C.5

D.-1

Answer -> A

Q15. Consider the Book class with a default constructor and the code snippet. What will be the output?

```
class Book {
    private String title;
    private double price;

    public Book() {
        this.title = "Unknown";
        this.price = 0.0;
    }

    public String getDetails() {
        return "Title: " + title + ", Price: " + price;
    }
}

Book b = new Book();
System.out.println(b.getDetails());
```

A.Title: null, Price: 0.0

B.Title: Unknown, Price: 0.0

C.Title: , Price: 0.0

D.Compilation Error

Answer -> B

Q16. Consider the Book class with a parameterized constructor and the code snippet. What will be the output?

```
class Book {
    private String title;
    private double price;
```

```
    public Book(String title, double price) {
        this.title = title;
        this.price = price;
    }

    public String getDetails() {
        return "Title: " + title + ", Price: " + price;
    }
}

Book b = new Book("Java Programming", 450.0);
System.out.println(b.getDetails());
```

A.Title: Java Programming, Price: 450.0

B.Title: null, Price: 0.0

C.Title: , Price: 0.0

D.Compilation Error

Answer -> A

Q17. Given the Wallet class and the code snippet, how many Wallet objects exist in memory after the execution of these lines?

```
class Wallet {
    int balance;
    public Wallet(int amount) {
        this.balance = amount;
    }
}

Wallet w1 = new Wallet(500);
Wallet w2 = w1; // w2 now refers to the same object as w1
Wallet w3 = new Wallet(500); // w3 refers to a new object
```

A.1

B.2

C.3

D.0

Answer -> B

Q18. Consider the Wallet class and the code snippet. What will be the output after the add() method is called through w2?

```
class Wallet {
    int balance;
    public Wallet(int amount) {
        this.balance = amount;
    }
    public void add(int amount) {
        this.balance += amount;
    }
}

Wallet w1 = new Wallet(500);
Wallet w2 = w1;
w2.add(200);

System.out.println("w1 balance: " + w1.balance);
System.out.println("w2 balance: " + w2.balance);
```

A.w1 balance: 500 / w2 balance: 700

B.w1 balance: 700 / w2 balance: 500

C.w1 balance: 700 / w2 balance: 700

D.w1 balance: 500 / w2 balance: 500

Answer -> C

Q19. Consider the Student class and the code snippet. What will be the output after s1.setName("Anu") is called?

```
class Student {
    String name;
    public Student(String name) {
        this.name = name;
    }
    public void setName(String newName) {
        this.name = newName;
    }
}

Student s1 = new Student("Asha");
Student s2 = new Student("Ravi");
s1.setName("Anu");

System.out.println("s1 name: " + s1.name);
System.out.println("s2 name: " + s2.name);
```

A.s1 name: Asha / s2 name: Ravi

B.s1 name: Anu / s2 name: Anu

C.s1 name: Anu / s2 name: Ravi

D.s1 name: Asha / s2 name: Anu

Answer -> C

Q20. Consider the Visitor class with a static visitorCount variable and the code snippet. What will be the output?

```
class Visitor {
    static int visitorCount = 0; // Shared by all visitors

    public Visitor() {
        visitorCount++;
    }
}

Visitor v1 = new Visitor();
Visitor v2 = new Visitor();
Visitor v3 = new Visitor();

System.out.println("Total visitors: " + Visitor.visitorCount);
```

A.Total visitors: 0

B.Total visitors: 1

C.Total visitors: 3

D.Total visitors: Error

Answer -> C

Part C — Concept Questions

Answer each question in a few sentences, giving reasons for your answer.

Question 1

Explain the concept of String immutability in Java and discuss its impact on performance when a String object undergoes repeated modifications. How can this performance impact be mitigated?

A String is immutable, meaning its value cannot be changed after creation. Methods such as `toUpperCase()` return a new string instead of modifying the original. Repeated modifications may create many temporary objects and reduce performance. Use `StringBuilder` when performing frequent string modifications.

Example:

```
String s = "java";
s = s.toUpperCase();
System.out.println(s);
```

Question 2

In a Java class constructor, when both an instance variable and a constructor parameter share the same name, how does the 'this' keyword resolve this ambiguity? Provide an example scenario to illustrate its use.

The `this` keyword refers to the current object. It distinguishes an instance variable from a constructor parameter when both have the same name.

Example:

```
class StudentInfo {
    String name;

    StudentInfo(String name) {
        this.name = name;
    }
}
```

Here, `this.name` refers to the instance variable, while `name` refers to the parameter.

Question 3

Differentiate between an object and a reference variable in Java. Explain what occurs in memory when one reference variable is assigned the value of another, detailing the implications for object manipulation.

An object is an instance of a class stored in memory. A reference variable holds a reference to that object. When one reference is assigned to another, both can refer to the same object, so changes made through either reference are visible through both.

Example:

```
Wallet w1 = new Wallet(500);
Wallet w2 = w1;
```

Only one Wallet object is created by these statements.

Question 4

Discuss the appropriate use cases for static variables and instance variables in Java. Provide a scenario where a static member would be preferred and another where an instance member is more suitable, justifying your choices.

A static variable belongs to the class and is shared by all its objects. An instance variable belongs to an individual object, so each object can have a different value.

- Static example: A visitor counter shared by all visitors.
- Instance example: Each student's name or marks.

Static variables are useful for shared information, while instance variables store object-specific information.

Question 5

Compare and contrast the for, while, and do-while loops in Java. Describe specific programming scenarios where each type of loop would be the most suitable choice, explaining your reasoning.

- For loop: Best when the number of iterations is known, such as printing numbers from 1 to 10.
- While loop: Best when repetition depends on a condition, such as reading values until a sentinel is entered.
- Do-while loop: Best when the code must execute at least once, such as displaying a menu before asking whether to continue.

A `for` loop checks its condition before each iteration, a `while` loop also checks before each iteration, and a `do-while` loop checks after each iteration.

Question 6

Explain the concept of pass-by-value in Java. Differentiate its behavior and implications when a method receives a primitive data type as an argument versus when it receives an object reference.

Java always uses pass-by-value. For primitive arguments, a copy of the value is passed, so modifying the parameter does not change the original variable. For object arguments, a copy of the reference is passed, so a method can modify the referenced object's fields, but reassigning the parameter does not change the caller's reference.

Example:

```
static void change(int n) {
    n = 10;
}
```

Calling `change(x)` does not change the original value of `x`.

Question 7

Describe how integer division operates in Java, specifically how it handles fractional results. Explain why an explicit type cast might be necessary to obtain a precise floating-point result from an arithmetic expression involving integers.

When both operands are integers, Java performs integer division and discards the fractional part. For example, `5 / 2` gives `2`. To obtain a floating-point result, at least one operand must be converted to a floating-point type.

Example:

```
System.out.println(5 / 2);          // 2
System.out.println((double) 5 / 2); // 2.5
```

Casting after integer division, such as `(double)(5 / 2)`, gives `2.0`, not `2.5`.

Question 8

Elaborate on the principle of encapsulation in Object-Oriented Programming. Discuss how Java's access modifiers (e.g., private) combined with getter and setter methods contribute to enforcing encapsulation and ensuring data integrity.

Encapsulation combines data and methods within a class while controlling access to the data. Using `private` variables prevents direct access from outside the class. Public getter and setter methods provide controlled access and can validate values before modifying them.

Example:

```
class StudentData {
    private int marks;
```

```
    public int getMarks() {
        return marks;
    }

    public void setMarks(int marks) {
        if (marks >= 0 && marks <= 100) {
            this.marks = marks;
        }
    }
}
```

This helps protect data integrity.

Question 9

Explain the fundamental distinction between using the == operator and the equals() method when comparing two objects in Java. Illustrate how overriding the equals() method affects object comparison.

For objects, == checks whether two references point to the same object. The `equals()` method checks logical equality according to its implementation. For example, two different String objects containing `"hello"` can be equal using `equals()` even though == returns false.

Example:

```
String s1 = new String("hello");
String s2 = new String("hello");

System.out.println(s1 == s2);      // false
System.out.println(s1.equals(s2)); // true
```

A class can override `equals()` to compare its objects based on their contents.

Question 10

Differentiate between method overloading and method overriding in Java, both of which are forms of polymorphism. Explain when each is resolved (compile-time vs. run-time) and provide a concise example for each concept.

Method overloading means using the same method name with different parameter lists in the same class. The compiler determines which overload to call at compile time.

Example:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Method overriding occurs when a subclass provides its own implementation of an inherited method with a compatible signature. The implementation to execute is determined at runtime based on the actual object.

Example:

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

Calling `sound()` on an `Animal` reference holding a `Dog` object prints `Bark`.